Feedback System Documentation

1. Project Overview

The **Feedback System** is a comprehensive web application designed for educational institutions to collect, manage, and analyze student feedback. It facilitates a streamlined process for students to provide feedback on their courses and instructors, while offering robust reporting tools for Heads of Departments (HODs) and Administrators.

Key Features

- **Role-Based Access Control**: Secure login for Students, HODs, and Administrators.
- **Student Feedback Portal**: Intuitive interface for students to submit feedback for their respective subjects.
- **Real-time Analytics**: HOD Dashboard providing immediate insights into feedback trends.
- **Comprehensive Reporting**: Generation of downloadable reports in **PDF**, **Excel**, and **Word** formats.
- **Feedback Restriction**: Mechanism to prevent duplicate submissions and manage feedback windows.
- **Responsive Design**: Optimized for desktops, tablets, and mobile devices.

---

2. System Architecture

Technology Stack

- **Frontend**: React.js (Vite), Tailwind CSS (or custom CSS), Framer Motion for animations.
- **Backend**: Node.js, Express.js.
- **Database**: MongoDB (Mongoose ODM).
- **Authentication**: JWT (JSON Web Tokens) with bcrypt for password hashing.
- **Reporting**: `jspdf` (PDF), `xlsx` (Excel), `docx` (Word).

Data Flow

1. **User Authentication**: Users log in via the Login Page. Credentials are verified against the MongoDB database.

2. **Role Routing**:

   - **Students** are routed to the Feedback Form.

   - **HODs** are routed to the HOD Dashboard.

   - **Admins** are routed to the Admin Dashboard.

3. **Data Submission**: Student feedback is sent to the backend API and stored in the `Feedbacks` collection.

4. **Data Retrieval**: Dashboards fetch aggregated data to display charts and tables.

---

3. User Modules

3.1 Student Module

• **Login**: Access via Roll Number and Password.

• **Feedback Form**:

- Displays list of subjects/faculty for the student's academic year and section.

- Questionnaire with rating capability (1-5 scale) and text comments.

- Validation to ensure all questions are answered.

• **Submission**: Single submission policy enforced per subject/semester.

3.2 HOD Module

• **Login**: Access via Email and Password.

• **Dashboard**:

- **Overview**: Total submissions, pending feedbacks.

- **Class-wise Analytics**: Breakdown of feedback by section/year.

- **Faculty Performance**: Aggregated scores for instructors.

• **Reports**:

- Generate detailed reports for specific classes or subjects.

- Export options: PDF (with charts/tables), Excel (raw data), Word (formatted document).

3.3 Admin Module

- **Login**: Secure admin access.
- **User Management**: Add/Edit/Delete Student and Faculty records.
- **Subject Management**: Map subjects to faculty and classes.
- **System Configuration**: reset feedback cycles, manage open/close dates.

---

4. API Documentation (Summary)

Authentication

- `POST /api/auth/login`: Authenticates user and returns JWT.

Feedback

- `POST /api/feedback/submit`: Submit new feedback.
- `GET /api/feedback/status/:studentId`: Check if student has already submitted.

Analytics (HOD/Admin)

- `GET /api/analytics/dashboard`: Get high-level stats.
- `GET /api/analytics/subject/:subjectId`: Get detailed feedback for a subject.
- `GET /api/analytics/faculty/:facultyId`: Get performance report for a faculty member.

---

5. Installation & Setup

Prerequisites

- Node.js (v14+)
- MongoDB (Local or Atlas)

Steps

1. **Clone Repository**:
   ```bash
   git clone <repository-url>
   cd FeedbackSystem
   ```

```

2. **Install Dependencies**:

   ```bash
   npm install
   ```

3. **Environment Variables**:

   Create a `.env` file in the root directory:

   ```env
   PORT=5000
   MONGO_URI=mongodb://localhost:27017/feedback_db
   JWT_SECRET=your_secret_key
   ```

4. **Run Application**:

   - Development: `npm run dev` (Frontend) & `npm run server` (Backend)
   - Production: `npm run build` & `npm start`

---

6. Deployment

The application is configured for deployment on **Vercel**.

• **Frontend**: Deployed via Vercel CLI or Git integration.

• **Backend**: Can be deployed as Serverless Functions (in `api/` directory) or a standalone server (e.g., Render/Heroku).

Generated by Feedback System - Department of Data Science
   ```